

Warrior Unreal 프로젝트 사용 기법 정리

Unreal Engine C++ 액션 RPG/서바이벌 전투 프로젝트의 핵심 구현 방식과 코드 근거 요약

작성일: 2026-05-30 | 기준 저장소: /Users/kim/Documents/GitHub/kht9544/Warrior_Unreal

이 문서는 현재 프로젝트의 Source/Warrior 및 Config 파일을 기준으로 사용된 주요 개발 기법을 정리한 것이다. 구현 세부는 Gameplay Ability System, Gameplay Tags, Enhanced Input, AI Perception/Behavior Tree, DataAsset 기반 설정, 컴포넌트 중심 캐릭터 구조, UI 이벤트 바인딩, 웨이브 스폰 시스템으로 나뉜다.

1. 프로젝트 구성과 사용 모듈

Warrior.Build.cs에는 GameplayTags, EnhancedInput, GameplayTasks, AnimGraphRuntime, MotionWarping, Niagara, NavigationSystem, MoviePlayer가 포함되어 있다. 즉 단순 캐릭터 조작을 넘어 GAS 기반 전투, 태그 중심 상태 관리, 모션 보정, 내비게이션 기반 AI/스폰, 연출/이펙트 확장을 염두에 둔 모듈 구성을 사용한다.

근거: Source/Warrior/Warrior.Build.cs

- Core/Engine/InputCore: 기본 Unreal C++ 게임플레이 기반
- GameplayTags + GameplayTasks: 태그 기반 입력·상태·이벤트와 AbilityTask 구현
- EnhancedInput: DataAsset과 GameplayTag를 연결한 입력 바인딩
- MotionWarping, AnimGraphRuntime: 전투 모션/애니메이션 확장
- NavigationSystem: AI 이동, 랜덤 스폰 위치 산출, 웨이브 시스템

2. GAS 기반 캐릭터 아키텍처

모든 캐릭터의 공통 부모인 AWarriorBaseCharacter는 UWarriorAbilitySystemComponent, UWarriorAttributeSet, UMotionWarpingComponent를 기본 서브오브젝트로 생성한다. PossessedBy에서 InitAbilityActorInfo(this, this)를 호출해 ASC의 Owner/Avatar 정보를 초기화한다.

근거: Source/Warrior/Public/Characters/WarriorBaseCharacter.cpp:19-23, 48-55

Hero와 Enemy는 공통 부모의 ASC/AttributeSet을 공유하면서 Combat/UI 컴포넌트만 각자 다르게 붙인다. 이는 캐릭터 종류별 차이는 컴포넌트로 분리하고, 어빌리티/속성/태그 처리의 공통 기반은 재사용하는 구조다.

근거: WarriorHeroCharacter.cpp:43-45, WarriorEnemyCharacter.cpp 생성자

적용 효과

- 전투 계산, 상태 태그, UI 갱신을 ASC/AttributeSet 중심으로 일관화
- Hero/Enemy의 구체 동작은 CombatComponent, UIComponent, GameplayAbility 파생 클래스로 분리
- Motion Warping을 캐릭터 기본 기능으로 넣어 공격/피격/이동 모션 보정 여지를 확보

3. GameplayTag 중심 설계

프로젝트는 입력, 플레이어 능력, 무기, 이벤트, 상태, SetByCaller 값, 적 능력/상태를 모두 GameplayTag로 명명한다. WarriorGameplayTags.cpp의 UE_DEFINE_GAMEPLAY_TAG 선언들이 전체 시스템의 공용 계약 역할을 한다.

근거: Source/Warrior/Public/WarriorGameplayTags.cpp

- InputTag.Move, InputTag.LightAttack.Axe 등 입력 구분
- Player.Ability.*, Enemy.Ability.*, Shared.Ability.*로 어빌리티 범주화
- Player.Event.HitPause, Shared.Event.MeleeHit 등 GameplayEvent 라우팅
- Shared.SetByCaller.BaseDamage, Player.SetByCaller.AttackType.*로 데미지 계산 인자 전달

이 방식은 C++ 클래스 간 직접 참조를 줄이고, Blueprint/GameplayEffect/GameplayAbility가 같은 태그 계약을 통해 연결되도록 만든다.

4. Enhanced Input + DataAsset 입력 매핑

UDataAsset_InputConfig는 UInputMappingContext와 NativeInputActions, AbilityInputActions 배열을 보유한다. 각 입력 액션은 GameplayTag와 함께 저장되며, UWarriorInputComponent의 템플릿 함수가 태그를 기준으로 Native 입력과 Ability 입력을 바인딩한다.

근거: DataAsset_InputConfig.h, WarriorInputComponent.h, WarriorHeroCharacter.cpp:101~114

HeroCharacter는 SetupPlayerInputComponent에서 DefaultMappingContext를 EnhancedInputLocalPlayerSubsystem에 추가하고, Move/Look은 즉시 C++ 함수에 바인딩한다. 어빌리티 입력은 Pressed/Released 콜백에 InputTag를 함께 전달하여 ASC가 해당 태그의 AbilitySpec을 활성화하거나 취소한다.

근거: WarriorAbilitySystemComponent.cpp:8~31

구현상 특징

- 입력 액션과 어빌리티가 클래스명이 아니라 GameplayTag로 연결된다.
- MustBeHeld 계열 입력은 Released 시 active ability를 취소하는 흐름을 갖는다.
- 무기 교체 시 WeaponInputMappingContext/DefaultWeaponAbilities를 확장할 수 있는 구조다.

5. DataAsset 기반 초기 능력/효과 주입

UDataAsset_StartUpDataBase는 시작 어빌리티와 시작 GameplayEffect를 ASC에 주입한다. Hero는 난이도가 쉬울수록 높은 AbilityApplyLevel을 적용하고, Enemy는 난이도가 어려울수록 높은 레벨을 적용한다.

근거: DataAsset_StartUpDataBase.cpp, WarriorHeroCharacter.cpp:64~95, WarriorEnemyCharacter.cpp:132~166

Hero는 CharacterStartUpData.LoadSynchronous()를 사용하고 Enemy는 UAssetManager의 RequestAsyncLoad를 사용한다. Enemy 쪽은 스폰 또는 소유 시점에 SoftObjectPath 기반 비동기 로딩으로 데이터 의존성을 늦게 해소한다.

의미

- 캐릭터별 시작 능력과 스탯 초기값을 코드가 아니라 에디터 데이터로 조정 가능
- 난이도에 따라 같은 Ability/Effect를 다른 레벨로 적용
- Soft reference와 StreamableManager를 사용해 로딩 결합도를 낮춤

6. 데미지 계산: Execution Calculation + SetByCaller

UGExeCalc_DamageTaken은 AttackPower, DefensePower, DamageTaken 속성을 캡처하고, GameplayEffectSpec의 SetByCallerTagMagnitudes에서 BaseDamage와 콤보 카운트를 읽어 최종 데미지를 계산한다.

근거: GExeCalc_DamageTaken.cpp:10-27, 54-124

최종 공식은 $\text{BaseDamage} * \text{SourceAttackPower} / \text{TargetDefensePower}$ 이며, 라이트 콤보는 $(\text{콤보수} - 1) * 5\%$, 헤비 콤보는 콤보수 * 15%의 추가 배율을 적용한다. 결과는 DamageTaken 속성에 Override Modifier로 출력된다.

$\text{FinalDamageDone} = \text{BaseDamage} * \text{SourceAttackPower} / \text{TargetDefensePower}$

AttributeSet은 DamageTaken이 실행되면 CurrentHealth를 차감하고 UI 델리게이트를 브로드캐스트한다. 체력이 0이 되면 Shared.Status.Dead 태그를 부여한다.

근거: WarriorAttributeSet.cpp:43-74

7. 전투 충돌과 GameplayEvent 라우팅

UPawnCombatComponent는 무기를 GameplayTag 키로 등록하고 현재 장착 무기의 BoxCollision을 QueryOnly/NoCollision으로 토글한다. 무기 충돌은 OnWeaponHitTarget, OnWeaponPulledFromTarget 델리게이트로 CombatComponent에 연결된다.

근거: PawnCombatComponent.cpp:7-27, 52-76

HeroCombatComponent는 이미 맞은 액터를 OverlappedActors로 중복 방지한 뒤 Shared.Event.MeleeHit와 Player.Event.HitPause를 자기 Pawn에게 전송한다. EnemyCombatComponent는 방어 태그와 unblockable 태그를 확인하고, 유효 방어면 Player.Event.SuccessfulBlock을 Target에게, 아니면 Shared.Event.MeleeHit을 적 자신에게 전송한다.

근거: HeroCombatComponent.cpp:27-50, EnemyCombatComponent.cpp:13-50

- 충돌 판정은 BoxComponent/Overlap 기반
- 어빌리티 트리거는 GameplayEvent로 연결
- 중복 히트 방지를 위해 OverlappedActors를 공격 윈도우 단위로 관리
- 방어 판정은 공격자/방어자 forward vector의 dot product를 이용

8. 무기 데이터와 동적 어빌리티 부여

FWarriorHeroWeaponData는 무기별 Anim Layer, Input Mapping Context, 기본 어빌리티, 특수 어빌리티, ScalableFloat 기반 WeaponBaseDamage, 아이콘 텍스처를 묶는다. UWarriorAbilitySystemComponent는 무기 장착 시 AbilitySpec에 InputTag를 DynamicAbilityTags로 추가해 부여하고, SpecHandle을 저장해 해제 시 제거한다.

근거: WarriorStructTypes.h, WarriorAbilitySystemComponent.cpp:38-69, WarriorHeroWeapon.cpp

즉 무기는 단순 Mesh가 아니라 입력, 애니메이션, 어빌리티, 데미지 스케일을 함께 갖는 데이터 단위로 설계되어 있다.

9. UI 이벤트 바인딩

UPawnUIComponent는 OnCurrentHealthChanged 델리게이트를 제공하고, UHeroUIComponent는 Rage 같은 Hero 전용 값을 확장한다. AttributeSet은 체력/분노 변화 후 UIComponent 델리게이트를 브로드캐스트한다.

근거: PawnUIComponent.h, WarriorAttributeSet.cpp:24-40

UWarriorWidgetBase는 NativeOnInitialized에서 소유 Pawn의 IPawnUIInterface를 통해 HeroUIComponent를 얻고 BlueprintImplementableEvent로 넘긴다. Enemy 체력 위젯은 InitEnemyCreateWidget에서 Enemy Actor를 받아 EnemyUIComponent를 전달한다.

근거: WarriorWidgetBase.cpp

- UI는 AttributeSet을 직접 폴링하지 않고 델리게이트 이벤트를 구독
- C++은 컴포넌트 전달과 이벤트 발생만 담당하고, 표시 구현은 Blueprint Widget에 위임

10. AI Perception, 팀 판정, Behavior Tree 확장

AWarriorAIController는 UCrowdFollowingComponent를 PathFollowingComponent로 사용해 Detour Crowd Avoidance를 활성화할 수 있게 구성한다. Sight Perception은 360도, 5000 범위로 적을 감지하고, 감지된 Actor를 Blackboard의 TargetActor에 기록한다.

근거: WarriorAIController.cpp:12-33, 36-64, 83-92

팀 판정은 GenericTeamAgentInterface의 GenericTeamId 비교로 구현된다. FunctionLibrary의 IsTargetPawnHostile도 QueryPawn/TargetPawn 컨트롤러의 TeamId를 비교한다.

근거: WarriorAIController.cpp:68-80, WarriorFunctionLibrary.cpp

Behavior Tree 확장으로는 TargetActor를 향해 매 Tick 보간 회전하는 Service와, 목표 각도 정밀도에 도달할 때까지 InProgress 상태로 유지되는 Rotate Task가 있다.

근거: BService_OrientToTargetActor.cpp, BTask_RotateToFaceTarget.cpp

11. 웨이브 스폰과 비동기 로딩

AWarriorSurvialGameMode는 WaitSpawnNewWave, SpawningNewWave, InProgress, WaveCompleted, AllWavesDone, PlayerDied 상태를 갖는 상태 머신으로 동작한다. BeginPlay에서 DataTable의 Row 수로 총 웨이브 수를 계산하고 다음 웨이브 적 클래스를 사전 로드한다.

근거: WarriorSurvialGameMode.h, WarriorSurvialGameMode.cpp:26-37, 39-90

웨이브 데이터는 DataTable Row 구조체 FWarriorEnemyWaveSpawnerTableRow로 관리된다. 각 Row는 적 SoftClass, 최소/최대 스폰 수, 총 스폰 수를 가진다. 스폰 위치는 레벨의 TargetPoint를 기준으로 NavigationSystem에서 랜덤 내비게이션 위치를 구해 사용한다.

근거: WarriorSurvialGameMode.cpp:104-129, 143-193

적이 파괴되면 OnDestroyed 델리게이트를 통해 카운터를 줄이고, 총량이 남아 있으면 추가 스폰, 현재 웨이브 적이 모두 사라지면 WaveCompleted로 전환한다.

근거: WarriorSurvialGameMode.cpp:202-230

12. 커스텀 AbilityTask

UAbilityTask_WaitSpawnEnemies는 특정 GameplayEvent를 기다린 뒤 SoftClass 적 클래스를 비동기 로드하고, NavigationSystem의 RandomReachablePointInRadius를 사용해 여러 적을 스폰한다. 결과는 OnSpawnFinished 또는 DidNotSpawn 델리게이트로 Ability Blueprint에 전달된다.

근거: AbilityTask_WaitSpawnEnemies.cpp

UAbilityTask_ExecuteTaskOnTick은 bTickingTask를 true로 설정하고 TickTask마다 DeltaTime을 브로드캐스트한다. Blueprint Ability 안에서 Tick 기반 보간, 추적, 지속 판정을 구현하기 위한 보조 태스크다.

근거: AbilityTask_ExecuteTaskOnTick.cpp

13. 유틸리티와 Blueprint 연동 기법

UWarriorFunctionLibrary는 ASC 획득, GameplayTag 추가/제거/검사, CombatComponent 획득, 적대 관계 판정, HitReact 방향 계산, 방어 유효성 판정, GameplayEffectSpec 적용, 입력 모드 전환, Latent Countdown을 제공한다.

근거: WarriorFunctionLibrary.cpp

- BlueprintCallable/BlueprintPure 래퍼로 C++ 로직을 Blueprint에서 재사용
- EWarriorValidType, EWarriorConfirmType 등 enum을 Exec pin처럼 써서 Blueprint 흐름을 명확화
- TWeakInterfacePtr 캐싱, Interface 기반 컴포넌트 접근으로 결합도 완화

14. 전체 구조 요약

이 프로젝트의 핵심은 GameplayTag를 공통 언어로 삼아 Enhanced Input, GameplayAbility, GameplayEffect, CombatComponent, UI, AI 이벤트를 연결한 점이다. 캐릭터는 ASC/AttributeSet을 공통 기반으로 갖고, 실제 역할 차이는 Hero/Enemy 전용 컴포넌트와 파생 Ability에서 처리한다.

데이터는 DataAsset과 DataTable로 분리되어 에디터 조정이 가능하며, SoftObject/SoftClass와 StreamableManager를 통해 필요한 클래스와 설정을 지연 로딩한다. AI와 웨이브 스폰은 NavigationSystem, Perception, Blackboard, Behavior Tree Task/Service를 조합해 구현되어 있다.

대표 기법 목록

- Unreal Gameplay Ability System 기반 어빌리티/속성/효과 구조
- GameplayTag 기반 입력, 상태, 이벤트, SetByCaller 데이터 계약
- Enhanced Input + DataAsset 입력 추상화
- ExecutionCalculation을 통한 공격력/방어력/콤보 기반 데미지 산출
- CombatComponent와 Box overlap 기반 공격 판정
- Delegate 기반 UI 갱신 및 Blueprint Widget 이벤트 연결
- AI Perception, GenericTeamAgentInterface, Detour Crowd Avoidance
- DataTable 기반 웨이브 상태 머신과 SoftClass 비동기 프리로드
- 커스텀 AbilityTask와 LatentAction을 통한 Blueprint 친화 확장

참고: 파일명에 Survial처럼 오타자로 보이는 이름이 있으나, 이 문서는 현재 저장소의 실제 파일명을 그대로 사용했다.